

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №1
по дисциплине «Web-технологии»
Тема: Тетрис на JavaScript.

Студент гр. 4342

Ларионов В.А.

Преподаватель

Беляев С.А.

Санкт-Петербург

2026

Постановка задачи

Необходимо создать web-приложение – игру в тетрис. Основные требования:

- сервер – nginx, протокол взаимодействия – HTTPS версии не ниже 2.0;
- отображается страница для ввода имени пользователя с использованием HTML-элементов `<input>`;
- статическая страница отображает «стакан» для тетриса с использованием HTML-элемента `<canvas>`, элемент `<div>` используется для отображения следующей фигуры, отображается имя пользователя;
- фигуры в игре – классические фигуры тетриса (7 шт. тетранино);
- случайным образом генерируется фигура и начинает падать в «стакан» (описание правил см., например, <https://ru.wikipedia.org/wiki/Тетрис>);
- пользователь имеет возможность двигать фигуру влево и вправо, повернуть на 90° и «уронить»;
- если собралась целая «строка», она должна исчезнуть;
- при наборе некоторого заданного числа очков увеличивается уровень, что заключается в увеличении скорости игры;
- пользователь проигрывает, когда стакан «заполняется», после чего отображается локальная таблица рекордов;
- вся логика приложения написана на JavaScript.

Выполнение работы

Были сгенерированы закрытый и открытый ключи командой `openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout example.key -out example.crt`.

Был настроен сервер nginx, конфигурация на Рисунке 1.

Для того, чтобы подключение было защищенным, http соединение с портом 80 перенаправляется на https соединение с портом 301. Далее обрабатывается https соединение версии 2 с помощью ssl. Также в параметрах `location` пользователя сервер перенаправляет на страницу с вводом имени.

```

events {}
http {
    server {
        listen 80;
        server_name localhost;
        return 301 https://$host$request_uri;
    }
    server {
        listen 443 ssl;
        server_name localhost;
        ssl_certificate example.csr;
        ssl_certificate_key example.key;

        location / {
            root .;
            index index.html;
        }
    }
}

```

Рисунок 1 – конфигурация nginx.

Ввод имени пользователя реализован на странице index.html с помощью элемента `<input>`. Пользователь вводит имя и нажимает кнопку «Играть». Введённое имя передаётся на страницу main.html через GET-параметр username. На странице игры JavaScript получает имя с помощью URLSearchParams. Ввод имени пользователя на Рисунке 2.



The image shows a black rectangular area representing a web page. At the top center, the word "ТЕТРИС" is written in large, white, bold, sans-serif capital letters. Below this, on the left side, the text "Имя пользователя:" is written in white. To its right is a white rectangular input field. Further to the right is a white rectangular button with the word "Играть" (Play) written in black text.

Рисунок 2 – страница с вводом имени пользователя.

На странице с игрой отображаются имя пользователя, количество очков, текущий уровень, игровое поле и следующая фигура. Игровое поле реализовано с помощью элемента `<canvas>`, следующая фигура отображается в отдельном `<canvas>`, расположенном внутри элемента `<div>`. В начале игры количество

очков равно 0, уровень равен 0. Игровое поле имеет размер 20×10 клеток. Основная страница Рисунок 3.

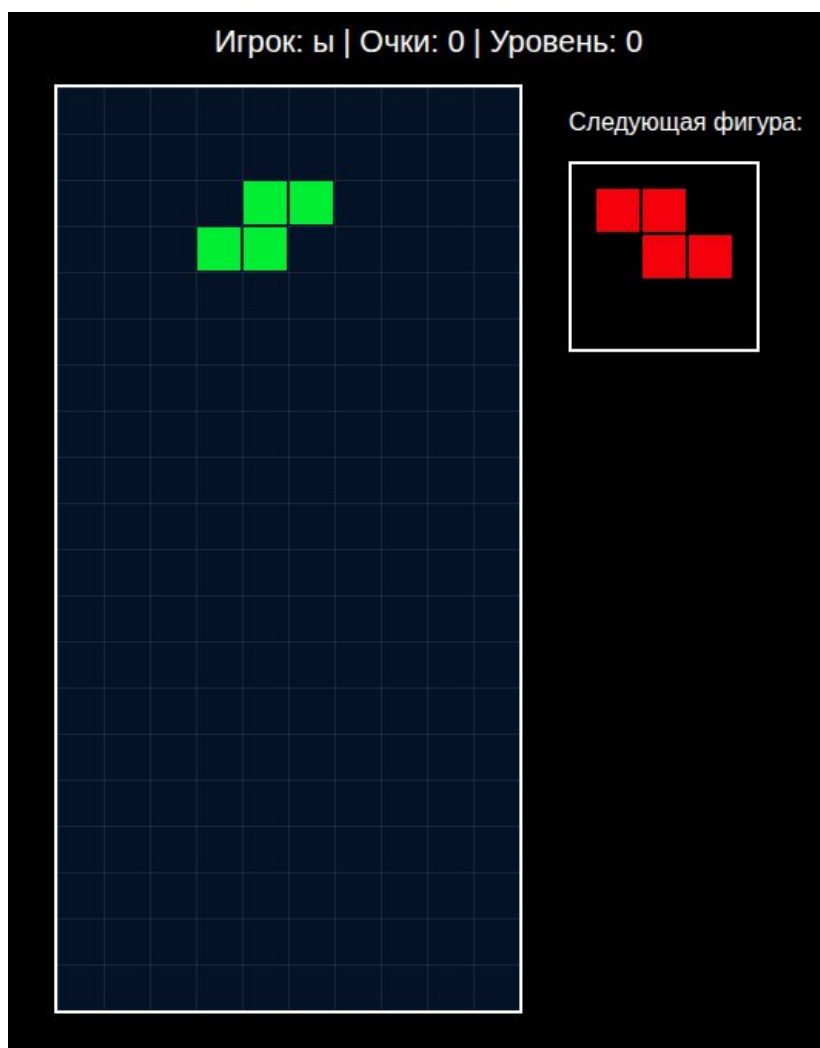


Рисунок 3 – страница с игрой.

Состояние игрового поля хранится в двумерном массиве `board`. Нулевое значение означает пустую клетку, ненулевое значение означает установленный блок. Текущая фигура хранится отдельно в двумерном массиве `currentMatrix`, а её положение на поле задаётся переменными `currentX` и `currentY`. Поэтому текущую фигуру можно перемещать и поворачивать до момента её установки на игровое поле.

Для создания фигур используется объект `SHAPES`, в котором заданы все 7 классических фигур тетриса: I, J, L, O, S, T и Z. Каждая фигура представлена двумерным массивом из нулей и единиц. Функция `randomType()` случайным

образом выбирает тип фигуры. После этого соответствующая матрица копируется в `currentMatrix`.

Движение фигуры реализовано функцией `move(dx, dy)`. Параметры `dx` и `dy` задают изменение координат по горизонтали и вертикали. Сначала вычисляются новые координаты фигуры, после чего функция `isCollision()` проверяет, можно ли установить фигуру в новом положении. Проверяется выход за левую и правую границы поля, выход за нижнюю границу и столкновение с уже установленными блоками. Если столкновение обнаружено, координаты фигуры не изменяются. Поворот фигуры реализован функцией `rotateMatrix()`, которая создаёт новую матрицу после поворота на 90 градусов. Функция `rotatePiece()` получает повернутую матрицу и проверяет её положение с помощью `isCollision()`. Если новая позиция допустима, текущая матрица заменяется повернутой. При невозможности поворота фигура остаётся в прежнем положении.

Обработка нажатий клавиш выполняется с помощью `document.addEventListener('keydown', ...)`. Клавиши `ArrowLeft` и `ArrowRight` перемещают фигуру влево и вправо, `ArrowDown` перемещает её вниз, `ArrowUp` выполняет поворот, а `Space` выполняет мгновенное падение фигуры.

Когда фигура больше не может перемещаться вниз, вызывается функция `lockPiece()`. Она записывает блоки текущей фигуры в массив `board` с учётом координат `currentX` и `currentY`. После этого вызывается функция `clearLines()`, которая проверяет строки игрового поля на заполненность. Если строка заполнена полностью, она удаляется, а сверху добавляется новая пустая строка. После очистки строк создаётся следующая фигура.

За удаление строк игрок получает очки. Количество очков рассчитывается по формуле `cleared * 100 * (level + 1)`, где `cleared` — количество удалённых строк. Уровень увеличивается после удаления каждых 10 строк. При повышении уровня уменьшается значение `dropInterval`, то есть сокращается интервал между автоматическими падениями фигур. При этом значение `dropInterval` не может стать меньше `MIN_SPEED`.

Окончание игры определяется при создании новой фигуры. После её появления программа проверяет, не пересекается ли она с уже установленными блоками. Если столкновение обнаружено, переменная `gameOver` устанавливается в `true` и вызывается функция `saveScore()`. Функция `saveScore()` загружает таблицу рекордов из `localStorage`, добавляет в неё имя игрока, количество очков и дату, сортирует результаты по убыванию количества очков и сохраняет первые 20 результатов обратно в `localStorage`. Для хранения данных используется формат JSON. После сохранения результата выполняется переход на страницу `scores.html`, где выводится таблица локальных рекордов.

Отрисовка игрового поля выполняется функцией `drawBoard()`. Сначала после отрисовки уже установленных блоков аналогичным образом рисуется текущая фигура. Для определения её положения используются `currentX` и `currentY`. Следующая фигура отрисовывается в отдельном Canvas функцией `drawNext()`. Игровой цикл реализован функцией `gameLoop()`. На каждом вызове сначала проверяется, закончилась ли игра. Затем выполняется попытка переместить текущую фигуру вниз. После этого перерисовываются игровое поле и следующая фигура, а также обновляются значения очков и уровня. Следующий вызов функции планируется с помощью `requestAnimationFrame()`. Интервал автоматического падения определяется переменной `dropInterval` и уменьшается при повышении уровня.

Вывод

В ходе лабораторной работы была разработана web-игра «Тетрис» на языке JavaScript. Реализованы ввод имени пользователя, случайное создание фигур, их перемещение и поворот, проверка столкновений, автоматическое и мгновенное падение, очистка заполненных строк, подсчёт очков, увеличение уровня и ускорение игры. При заполнении игрового поля определяется окончание игры, после чего результат сохраняется в `localStorage` и отображается в локальной таблице рекордов.